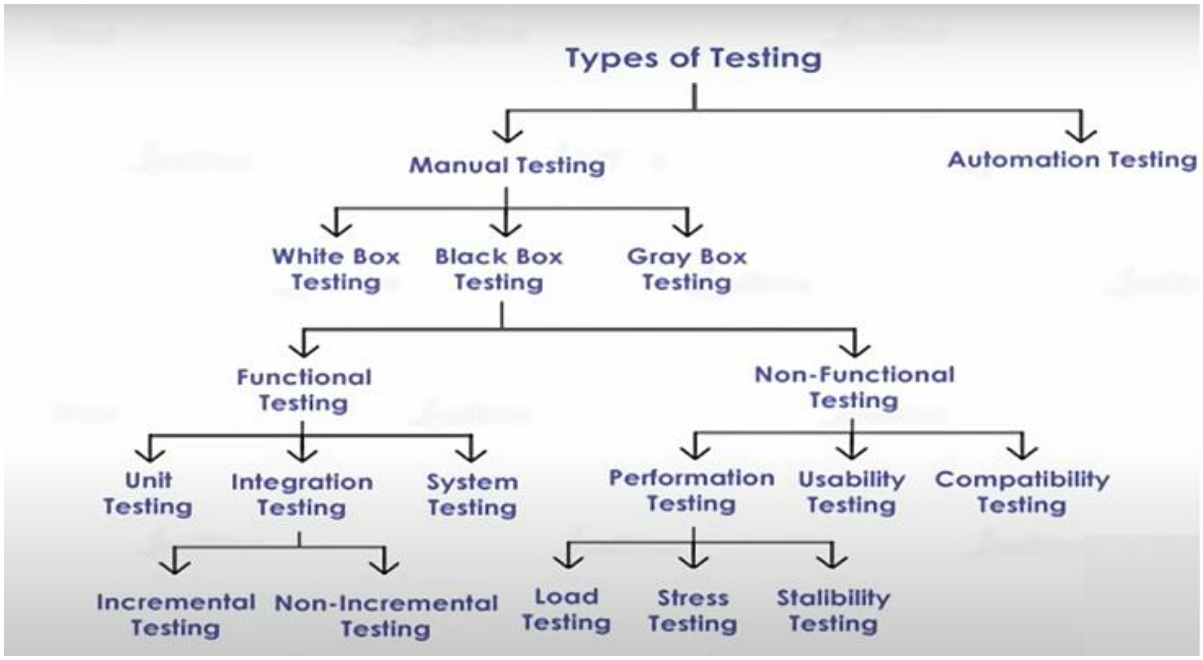
Module 1_Part B : Software Testing (Test Case Design)

To check whether the Actual software product matches the Expected requirements and to ensure that software product is defect free.

Id	Input	Expected Result	Actual Result	Status
1	Read ATM Card	Accept card and ask for pin #	Accepted the card and asked for a pin.	Passed
2	Read Invalid Card	"No ATM Card" exception is thrown and card is returned to the user.	Accepted the card and asked for a pin.	Failed
3	Invalid PIN Entered	"Stolen Card" exception is thrown and card is destroyed.	Accepted the card and asked for a pin.	Failed

Types of Software Testing



Black Box Testing

In Black Box Testing, the functionalities of software applications are tested without having knowledge of Internal code structure, Implementation details and Internal paths.

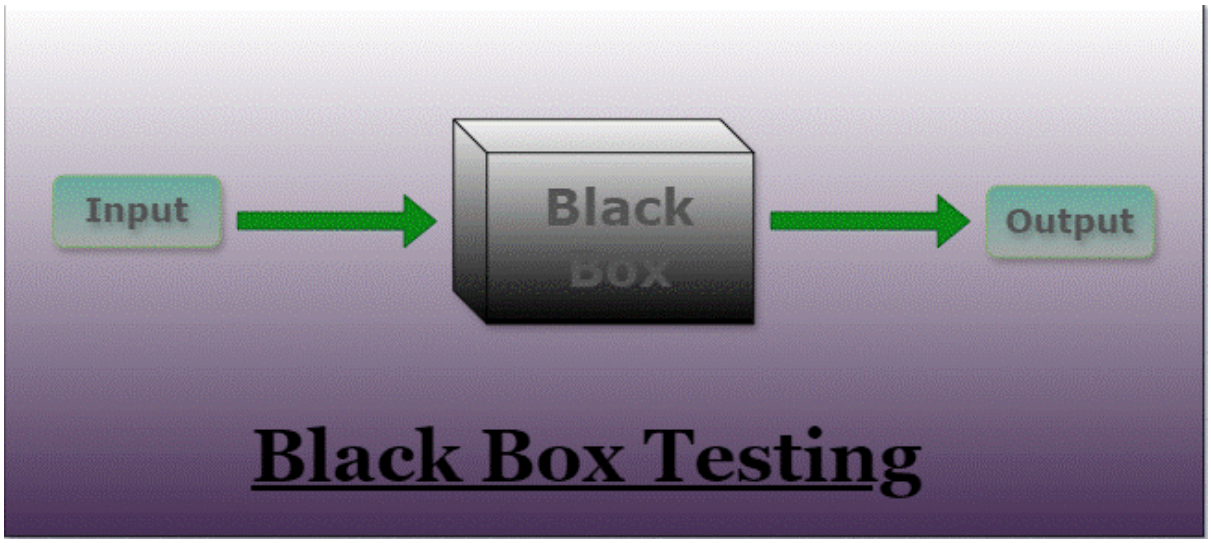
• Tester gives input value to examine its functionality & checks whether function is giving expected output or not.

• If the function produces correct output, then it is passed in testing, otherwise failed.

• It is also known as Behavioral Testing, Functional & Closed box Testing.

• Perform by the Software Testers.

• Black Box Testing Tools: QTP, Selenium, Loadrunner & Jmeter



Types Of Black Box Testing

The following are the several categories of black box testing:

1. **Functional Testing**
2. **Regression Testing**
3. **Nonfunctional Testing (NFT)**

Before we move in depth of the Black box testing do you known that there are many different type of testing used in industry and some automation testing tools are there which automate the most of testing so if you wish to learn the latest industry level tools then you check-out our manual to automation testing course in which you will learn all these concept and tools

Functional Testing

- Functional testing is defined as a type of testing that verifies that each function of the software application works in conformance with the requirement and specification.
- This testing is not concerned with the source code of the application. Each functionality of the software application is tested by providing appropriate test input, expecting the output, and comparing the actual output with the expected output.
- This testing focuses on checking the user interface, APIs, database, security, client or server application, and functionality of the Application Under Test. Functional testing can be manual or automated. It determines the system's software functional requirements.

Regression Testing

- Regression Testing is the process of testing the modified parts of the code and the parts that might get affected due to the modifications to ensure that no new errors have been introduced in the software after the modifications have been made.

- Regression means the return of something and in the software field, it refers to the return of a bug. It ensures that the newly added code is compatible with the existing code.
- In other words, a new software update has no impact on the functionality of the software. This is carried out after a system maintenance operation and upgrades.

Nonfunctional Testing

- Non-functional testing is a software testing technique that checks the non-functional attributes of the system.
- Non-functional testing is defined as a type of software testing to check non-functional aspects of a software application.
- It is designed to test the readiness of a system as per nonfunctional parameters which are never addressed by functional testing.
- Non-functional testing is as important as functional testing.
- Non-functional testing is also known as NFT. This testing is not functional testing of software. It focuses on the software's performance, usability, and scalability.

Advantages of Black Box Testing

- The tester does not need to have more functional knowledge or programming skills to implement the Black Box Testing.
- It is efficient for implementing the tests in the larger system.
- Tests are executed from the user's or client's point of view.
- Test cases are easily reproducible.
- It is used to find the ambiguity and contradictions in the functional specifications.

Disadvantages of Black Box Testing

- There is a possibility of repeating the same tests while implementing the testing process.
- Without clear functional specifications, test cases are difficult to implement.
- It is difficult to execute the test cases because of complex inputs at different stages of testing.
- Sometimes, the reason for the test failure cannot be detected.
- Some programs in the application are not tested.
- It does not reveal the errors in the control structure.
- Working with a large sample space of inputs can be exhaustive and consumes a lot of time.

Black Box Techniques

Random Testing

Random testing is software testing in which the system is tested with the help of generating random and independent inputs and test cases. Random testing is also named monkey testing. It is a black box assessment outline technique in which the tests are being chosen randomly and the results are being compared by some software identification to check whether the output is correct or incorrect.

Working Random Testing:

Step-1: Identify Input domain

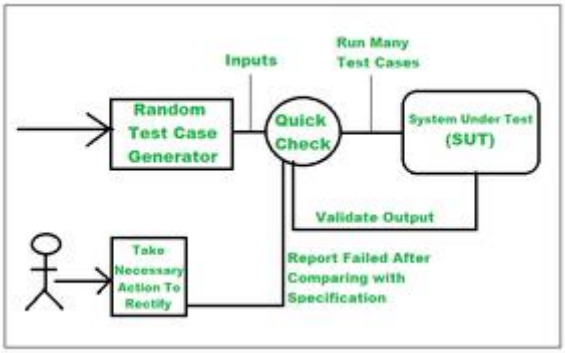
Step-2: Select test inputs independently/randomly from the input domain

Step-3: Test the system on these inputs and form a random test set

Step-4: Compare the result with system specification

Step-5: If the Report fails then take necessary action.

The below image represents the working of Random Testing more clearly.



Characteristics of Random Testing:

1. Random testing is implemented when the bug in an application is not recognized.
2. It is used to check the system's execution and dependability.
3. It saves our time and does not need any extra effort.
4. Random testing is less costly, it doesn't need extra knowledge for testing the program.

Methods to Implement Random Testing:

To implement the random testing basically, four steps are applied:

1. The user input domain is analyzed.
2. After that, from that domain, the data of test inputs are chosen separately.
3. With the help of these test inputs, the test is executed successfully. These input tests conduct random sets of tests.

4. The outcomes are compared with the system identification. The outcome of the test becomes unsuccessful if any test input doesn't match with the original one otherwise the outcomes are always successful.

Advantages of Random Testing

1. It is very cheap so that anyone can use this software.
2. It doesn't need any special intelligence to access the program during the tests.
3. Errors can be traced very easily; it can easily detect the bug throughout the testing.
4. This software is lacking bias means it makes the groups evenly for the testing and it prefers not to repeatedly check the errors as there can be some changes in the codes throughout the testing process.

Disadvantages of Random Testing

1. This software only finds changes errors.
2. They are not practical. Some tests will be of no use for a longer time.
3. Most of the time is consumed by analyzing all the tests.
4. New tests cannot be formed if their data is not available during testing.

Example:

Scenario-Based Question: Random Testing

Scenario:

You are testing a new mobile banking application. The app includes several features such as viewing account balances, transferring funds, paying bills, and checking transaction history. To ensure robustness, you decide to use random testing to identify any potential issues that might not be captured by other testing methods.

Question:

Design test cases using Random Testing to validate the functionality and stability of the mobile banking application. Specify the expected results for each test case.

Answer:

Test Cases:

1. **Test Case 1: Random Transfer Amount**
    - **Input: Transfer a randomly selected amount (e.g., $42.73) from one account to another.**
    - **Expected Result: The transfer should be processed correctly, with the appropriate amount deducted from the source account and added to the destination account. The transaction should appear in both accounts' transaction history.**
2. **Test Case 2: Random Account Balance Inquiry**
    - **Input: Check the balance of a randomly selected account (e.g., Account #2315).**

- **Expected Result: The balance displayed should be accurate and match the expected amount based on recent transactions. The app should display the balance correctly without any discrepancies.**

3. **Test Case 3: Random Bill Payment**
    - **Input: Pay a randomly selected bill (e.g., electricity bill) for a randomly selected amount (e.g., $75.50).**
    - **Expected Result: The bill payment should be processed successfully, with the amount deducted from the account. The payment confirmation should be displayed, and the bill should be marked as paid in the app's bill payment section.**
4. **Test Case 4: Random Transaction History Check**
    - **Input: View transaction history for a randomly selected date range (e.g., from March 1 to March 7).**
    - **Expected Result: The transaction history should display all transactions accurately for the selected date range. The details of each transaction should be correct and reflect the activity within that period.**
5. **Test Case 5: Random App Navigation**
    - **Input: Navigate randomly through different sections of the app (e.g., from account overview to transaction history to bill payments).**
    - **Expected Result: The app should navigate smoothly between sections without errors or crashes. All features and options should function correctly as the user navigates through the app.**
6. **Test Case 6: Random User Interaction**
    - **Input: Perform a series of random actions within the app (e.g., login, logout, change account settings, and log in again).**
    - **Expected Result: The app should handle each action correctly, without any unexpected behavior or errors. The login and logout processes should work smoothly, and changes in account settings should be saved and reflected.**
7. **Test Case 7: Random Error Handling**
    - **Input: Simulate random error scenarios (e.g., network failure during a fund transfer).**
    - **Expected Result: The app should display appropriate error messages and handle the errors gracefully. It should provide options to retry or cancel the operation and ensure that no inconsistent state is left.**

Explanation:

Random Testing involves executing random actions or inputs to discover potential issues that might not be identified through structured testing methods. In this scenario:

- **Random Transfer Amount, Account Balance Inquiry, Bill Payment: Tests various functions with randomly chosen inputs to ensure correct processing and accuracy.**
- **Random Transaction History Check: Ensures that historical data is accurately displayed for a random date range.**
- **Random App Navigation and User Interaction: Validates the app's stability and correct behavior through random navigation and actions.**
- **Random Error Handling: Checks how the app handles unexpected error scenarios.**

By performing these random tests, you can uncover issues that might not be revealed through other testing strategies, contributing to a more robust and reliable application.

Equivalence Partitioning Technique

Here, input values that provide to system are divided into different classes or groups based on its similarity in the outcome.

Instead of using each and every input value, use any one value from the group to test outcome**.**

Scenario:

You are testing an online registration form that includes a field for users to enter their age. The age must be between 18 and 65 inclusive. You need to design test cases to validate this age input field using Equivalence Class Partitioning.

Question:

Design test cases using Equivalence Class Partitioning to ensure that the age input field is correctly validated. Specify the expected results for each test case.

Answer:

Equivalence Classes:

1. **Valid Age Class: Ages between 18 and 65 inclusive.**
2. **Invalid Age Class - Below Minimum: Ages less than 18.**
3. **Invalid Age Class - Above Maximum: Ages greater than 65.**

Test Cases:

1. **Test Case 1: Age in Valid Range**
    - **Input: 25**
    - **Expected Result: The system should accept the input and proceed with the registration.**

2. **Test Case 2: Age Just Below Minimum**
    - **Input: 17**
    - **Expected Result: The system should reject the input and display an error message indicating that the age must be between 18 and 65.**
3. **Test Case 3: Age Just Above Maximum**
    - **Input: 66**
    - **Expected Result: The system should reject the input and display an error message indicating that the age must be between 18 and 65.**

Explanation:

Equivalence Class Partitioning divides the input data into classes where all values within a class are expected to be treated the same way. For this scenario:

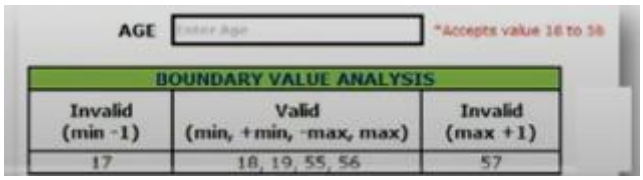
- **Valid Age Class (18-65): Includes any age that falls within this range. Only one test case is needed from this class to confirm that the system accepts valid inputs.**
- **Invalid Age Class - Below Minimum (<18): Includes all ages less than 18. Testing with a value just below the minimum boundary (17) is sufficient to ensure that inputs in this range are rejected.**
- **Invalid Age Class - Above Maximum (>65): Includes all ages greater than 65. Testing with a value just above the maximum boundary (66) is sufficient to ensure that inputs in this range are rejected.**

By selecting representative test cases from each class, you effectively validate that the application handles all possible input scenarios within the defined boundaries.

Boundary Value Analysis

It tests, boundary values are those that contain the upper and lower limit of a variable.

It tests, while entering boundary value whether the software is producing correct output or not.



You are testing an online registration form for a website. The form includes a field for users to enter their age. The age must be between 18 and 65 inclusive. You need to design test cases to validate this age input field using Boundary Value Analysis.

Question:

Design test cases using Boundary Value Analysis to ensure that the age input field is correctly validated. Specify the expected results for each test case.

Answer:

Test Cases:

1. **Test Case 1: Age at the Lower Boundary**
    - **Input: 17**
    - **Expected Result: The system should reject the input and display an error message indicating that the age must be between 18 and 65.**
2. **Test Case 2: Age at the Lower Boundary (Valid)**
    - **Input: 18**
    - **Expected Result: The system should accept the input and proceed with the registration.**
3. **Test Case 3: Age Just Above the Lower Boundary**
    - **Input: 19**
    - **Expected Result: The system should accept the input and proceed with the registration.**
4. **Test Case 4: Age Just Below the Upper Boundary**
    - **Input: 64**
    - **Expected Result: The system should accept the input and proceed with the registration.**
5. **Test Case 5: Age at the Upper Boundary**
    - **Input: 65**
    - **Expected Result: The system should accept the input and proceed with the registration.**
6. **Test Case 6: Age Just Above the Upper Boundary**
    - **Input: 66**
    - **Expected Result: The system should reject the input and display an error message indicating that the age must be between 18 and 65.**

Explanation:

Boundary Value Analysis focuses on testing values at the edges of input ranges. In this case, the boundaries are 18 and 65. The test cases include values just below the lower boundary (17), at the lower boundary (18), just above the lower boundary (19), just below the upper boundary (64), at the upper boundary (65), and just above the upper boundary (66). This approach helps ensure that the age validation logic handles boundary conditions correctly.

Decision Table Testing

Various input combination & their respective system behaviour are captured in tabular form.

Check logical relationship between two and more than two inputs. Ex. Gmail Account

Email (condition1)	T	T	F	F
Password (condition2)	T	F	T	F
Expected Result (Action)	Account Page	Incorrect password	Incorrect email	Incorrect email

Scenario-Based Question: Decision Table Testing for Gmail Login and Password

Scenario:

You are testing the Gmail login system. The system behavior is based on two main conditions:

Condition 1: The entered email/username is correct or incorrect.

Condition 2: The entered password is correct or incorrect.

The system should behave as follows:

If both the email and password are correct, login should be successful.

If either the email or the password is incorrect, login should fail with an appropriate error message.

Question: Create a decision table based on the login behavior for Gmail and derive test cases to ensure the system works as expected. Specify the expected results for each test case.

Decision Table:

Decision Table:

Test Case	Email Correct	Password Correct	Expected Outcome
1	Yes	Yes	Login Successful
2	Yes	No	Login Failed: Incorrect Password
3	No	Yes	Login Failed: Incorrect Email
4	No	No	Login Failed: Incorrect Email and Password

Test Case 1: Valid Email and Valid Password

Input: Enter correct email (e.g., "john.doe@gmail.com") and correct password (e.g., "Password123").

Expected Result: The login should be successful. The user should be taken to the Gmail inbox.

Test Case 2: Valid Email and Incorrect Password

Input: Enter correct email (e.g., "john.doe@gmail.com") and incorrect password (e.g., "WrongPass123").

Expected Result: The login should fail with an error message stating, "Wrong password. Try again or click 'Forgot password' to reset it."

Test Case 3: Incorrect Email and Valid Password

Input: Enter incorrect email (e.g., "jane.doe@gmail.com") and correct password (e.g., "Password123").

Expected Result: The login should fail with an error message stating, "Couldn't find your Google Account."

Test Case 4: Incorrect Email and Incorrect Password

Input: Enter incorrect email (e.g., "fake.email@gmail.com") and incorrect password (e.g., "FakePass123").

Expected Result: The login should fail with an error message stating, "Couldn't find your Google Account." The system should not distinguish between incorrect email and password for security reasons.

Explanation:

Decision Table Testing helps to identify all possible combinations of conditions (in this case, correct or incorrect email and password) and their expected outcomes.

Each row in the decision table represents a unique combination of the conditions (correct or incorrect email and password) and their respective results (successful or failed login).

The test cases ensure that the system handles each scenario correctly, covering both successful login and various failure scenarios with appropriate error messages.

By using decision table testing, you can comprehensively verify the Gmail login functionality, ensuring that the system behaves as expected for all possible input combinations.

- ## Error Guessing
    - It is based on the experience of the tester, where tester uses experience to guess the problematic areas of the software.
    - Examples: Divide by zero, Handling null values in the text fields, Accepting the submit button without any value, File upload without attachment, file upload with less than or more that the limit size.

    Scenario-Based Question: Error Guessing Testing

    Scenario:

    You are testing an online form for a travel booking website where users can book flights. The form has the following fields:

Full Name: Must contain only alphabetic characters.

Email Address: Must follow a valid email format (e.g., example@domain.com).

Travel Dates: Must be in the future.

Number of Passengers: Must be a positive integer (up to 9).

Credit Card Number: Must be exactly 16 digits long.

You are tasked with using error guessing to identify potential errors users might make when filling out this form, based on common mistakes you anticipate.

Question: Identify possible test cases using the Error Guessing technique to test the online form. List the errors you would guess and the expected outcomes for each test case.

Test Cases Using Error Guessing:

Test Case 1: Invalid Characters in Full Name

Input: Enter "John123" in the "Full Name" field.

Expected Result: The system should display an error message stating that the name must contain only alphabetic characters. The form should prevent submission.

Test Case 2: Invalid Email Format (Missing '@')

Input: Enter "johndoe.com" in the "Email Address" field.

Expected Result: The system should display an error message indicating that the email address is invalid. The form should prevent submission.

Test Case 3: Travel Date in the Past

Input: Enter a travel date of "January 15, 2023" (assuming today's date is September 2024).

Expected Result: The system should display an error message stating that the travel date must be in the future. The form should prevent submission.

Test Case 4: Negative Number of Passengers

Input: Enter "-2" in the "Number of Passengers" field.

Expected Result: The system should display an error message stating that the number of passengers must be a positive integer. The form should prevent submission.

Test Case 5: Exceeding Maximum Number of Passengers

Input: Enter "12" in the "Number of Passengers" field.

Expected Result: The system should display an error message indicating the maximum allowed number of passengers is 9. The form should prevent submission.

Test Case 6: Empty Credit Card Field

Input: Leave the "Credit Card Number" field blank and attempt to submit the form.

Expected Result: The system should display an error message indicating that the credit card number is required. The form should prevent submission.

Test Case 7: Invalid Credit Card Number (Too Short)

Input: Enter "12345678901234" (14 digits) in the "Credit Card Number" field.

Expected Result: The system should display an error message indicating that the credit card number must be exactly 16 digits long. The form should prevent submission.

Test Case 8: Invalid Travel Date Format

Input: Enter "15/2024/09" (incorrect date format) in the "Travel Dates" field.

Expected Result: The system should display an error message indicating the correct date format (e.g., MM/DD/YYYY). The form should prevent submission.

Test Case 9: Special Characters in Full Name

Input: Enter "John@Doe" in the "Full Name" field.

Expected Result: The system should display an error message stating that the name must contain only alphabetic characters. The form should prevent submission.

Test Case 10: Multiple Errors in One Submission

Input: Enter "John123" in the "Full Name" field, "johndoe.com" as the email address, and "-2" as the number of passengers.

Expected Result: The system should display all relevant error messages (for the invalid name, email format, and negative passenger count) and prevent the form from being submitted.

Explanation:

Error Guessing relies on the tester's experience and intuition to guess common mistakes that users might make. In this case, the errors include invalid input formats, missing mandatory fields, and values that violate the constraints.

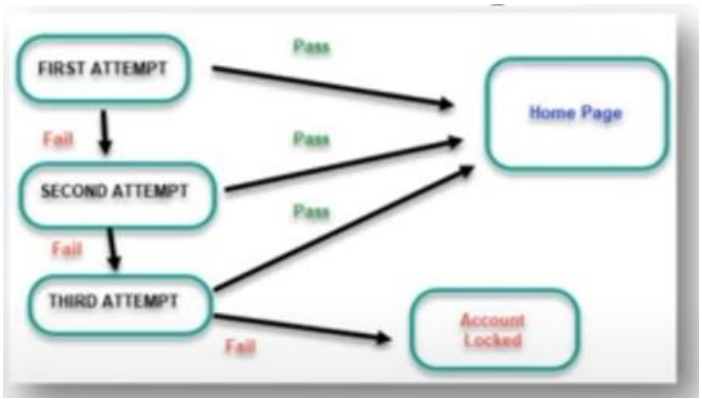
The test cases ensure that the system properly handles invalid inputs and provides useful error messages, preventing incorrect data from being submitted.

By anticipating common mistakes and testing these potential errors, you can identify weaknesses in the form validation logic and improve the overall robustness of the system.

State Transitioning Testing

It is used to capture behaviour of software application when different input values are given to the same function.

Applies to those types of application that provide specific number of attempts to access application.



Scenario-Based Question: State Transition Testing

Scenario:

You are testing the order management system for an e-commerce platform. The system has an order lifecycle with the following states:

1. **State 1: Order Placed**
2. **State 2: Payment Processed**
3. **State 3: Shipped**
4. **State 4: Delivered**
5. **State 5: Canceled**

The transitions between these states are defined as follows:

1. **From Order Placed to Payment Processed (after payment is successfully processed).**
2. **From Payment Processed to Shipped (after the order is packed and shipped).**
3. **From Shipped to Delivered (after the order is delivered to the customer).**
4. **From Order Placed to Canceled (if the order is canceled before payment).**
5. **From Payment Processed to Canceled (if the order is canceled after payment but before shipping).**
6. **Orders cannot be transitioned back from Shipped, Delivered, or Canceled to any previous state.**

Question:

Design test cases using State Transition Testing to validate the transitions between these states. Specify the expected results for each test case.

Answer: Test Cases Using State Transition Testing:

1. **Test Case 1: Transition from Order Placed to Payment Processed**
    - **Input: Place an order, then process the payment.**
    - **Initial State: Order Placed**

- Action: Process payment successfully.
- Expected Result: The order state should transition to Payment Processed. Verify that the payment is recorded, and the system reflects the new state.

2. Test Case 2: Transition from Payment Processed to Shipped
   - Input: After processing the payment, mark the order as shipped.
   - Initial State: Payment Processed
   - Action: Update the order status to Shipped.
   - Expected Result: The order state should transition to Shipped. Verify that the shipping information is recorded, and the system reflects the new state.
3. Test Case 3: Transition from Shipped to Delivered
   - Input: After the order has been shipped, mark it as delivered.
   - Initial State: Shipped
   - Action: Update the order status to Delivered.
   - Expected Result: The order state should transition to Delivered. Verify that the delivery information is recorded, and the system reflects the new state.
4. Test Case 4: Transition from Order Placed to Canceled
   - Input: Place an order and then cancel it before processing the payment.
   - Initial State: Order Placed
   - Action: Cancel the order before payment.
   - Expected Result: The order state should transition to Canceled. Verify that the cancellation is recorded, and no further actions (like payment or shipping) are possible.
5. Test Case 5: Transition from Payment Processed to Canceled
   - Input: After processing the payment, cancel the order before shipping.
   - Initial State: Payment Processed
   - Action: Cancel the order before marking it as shipped.
   - Expected Result: The order state should transition to Canceled. Verify that the cancellation is recorded and no further actions (like shipping or delivery) are possible.
6. Test Case 6: Invalid Transition from Delivered to Previous States
   - Input: Attempt to transition an order from Delivered back to Order Placed or Payment Processed.
   - Initial State: Delivered
   - Action: Try to update the state to Order Placed or Payment Processed.

- **Expected Result: The system should prevent this invalid transition and display an appropriate error message indicating that transitions from Delivered to previous states are not allowed.**

7. **Test Case 7: Invalid Transition from Canceled to Previous States**
    - **Input: Attempt to transition an order from Canceled back to Order Placed or Payment Processed.**
    - **Initial State: Canceled**
    - **Action: Try to update the state to Order Placed or Payment Processed.**
    - **Expected Result: The system should prevent this invalid transition and display an appropriate error message indicating that transitions from Canceled to previous states are not allowed.**
8. **Test Case 8: Valid Transition Path**
    - **Input: Place an order, process payment, mark it as shipped, and finally, deliver it.**
    - **Initial State: Order Placed**
    - **Action: Process payment, ship the order, and mark it as delivered.**
    - **Expected Result: The order should transition through each state (Order Placed → Payment Processed → Shipped → Delivered) correctly, and each state change should be reflected in the system.**

Explanation:

- **State Transition Testing involves validating that the system correctly transitions between states based on predefined rules.**
- **Each test case ensures that the system correctly handles valid transitions and prevents invalid ones.**
- **Testing transitions helps verify that the state management logic in the system is functioning correctly and that invalid state changes are appropriately handled.**

By using state transition testing, you can ensure that the order management system adheres to the defined state transitions and accurately reflects the order lifecycle.